

Task 3: Code Quality and Architecture

Ajibola Akelebe · Frontend Developer, Branda V2 · October 2026

Branda V2 has three jobs: let customers order, help them find Branda through search, and give staff and clients dashboards to manage the work. I would keep all three in one Next.js app with clear boundaries, so a change in one doesn't break the others.

Folder structure

```
src/
  app/
    [market]/           /ng /us /uk /ca
      (shop)/           home, services, cart, checkout
      (content)/        guides, case studies, blog (the SEO layer)
      (dashboard)/      client, production and admin dashboards, behind login
    api/               payment webhooks (Paystack, Stripe, PayPal)
    sitemap.ts, robots.ts
  components/
    ui/               buttons, inputs, dialogs: the shared design system
    cart/, catalog/, ... feature components
  lib/               data access, pricing, market config, validation
```

Route groups `(shop)`, `(content)` and `(dashboard)` each get their own layout without changing the URL. The `[market]` folder means every public page exists once per market, so `/ng/services` and `/uk/services` are the same code with different data.

Components

- Start specific. Extract a shared component when the third copy appears, not before.
- Keep props small and clear. One `variant` prop beats five true/false flags.
- Use native HTML first (`<dialog>`, `<select>`, radio inputs), so keyboard support and accessibility come for free.
- Keep shared logic in one place. In this build one `orderSummary()` function feeds the cart drawer, cart page and checkout, so the totals can never disagree.

State: server vs client

- **Server state** (catalogue, prices, orders, the logged-in user) is loaded in Server Components, and in TanStack Query where a dashboard needs live updates.
- **Client state** is kept small. The cart lives in Zustand, saved to the browser. Small UI state stays in `useState`.
- **Filters live in the URL.** Any search or filter result can be shared, bookmarked and indexed. That's how the Branda catalogue works now.

API and service layer

- Components never call `fetch` directly. Each resource has a small module in `lib/` (`getServices`, `createOrder`).
- Data from outside is checked with Zod at the boundary. A bad response fails loudly in one place instead of quietly breaking the UI.
- Secret keys and private calls stay in server-only code.

Forms and validation

- React Hook Form with Zod for anything bigger than a few fields.
- The same Zod schema runs again on the server before anything is saved. The browser check is for the user's convenience; the server check is the one that counts.
- For simple forms, native HTML validation (`required`, `type="email"`) is enough. Branda's checkout uses it.

Error handling

- An `error.tsx` per section, so a broken catalogue doesn't take down the header or the cart.
- `not-found.tsx` and designed empty states, each with a clear next step.
- Production errors are reported to Sentry or GlitchTip (I've used both), with enough context to reproduce them.

Authentication and roles

- Google sign-in plus email, with Clerk, Auth.js or Supabase Auth. I've shipped Clerk and Supabase Auth before.
- Each user has a role in their session: client, production staff or admin.
- `proxy.ts` (Next 16's new name for middleware) redirects people away from dashboards they can't see, early.
- Every page and action checks the role again on the server before loading data. The proxy alone isn't enough.

Markets, currency and localisation

- One config per market: currency, tax, delivery threshold, headline and featured services. Adding a market means adding one entry.
- Prices are formatted with `Intl.NumberFormat`, so ¥ , \$, £ and CA\$ follow local rules.
- Every page sets a canonical URL, `hreflang` links to the other three markets plus `x-default`, and localised titles. The sitemap lists every market version.
- If Canada needs French later, a language segment can sit under the market without moving anything else.

Responsive design

Mobile first, with Tailwind breakpoints and fluid type (`clamp`) for large headings. I design the small screen first, then let it grow. I spent five years as a designer before engineering, so I check the 390px view as carefully as the desktop one.

Maintainability, testing and docs

- Strict TypeScript, no `any` .
- Unit tests for anything involving money: prices, tax, totals. This repo has one (`pnpm test`).
- End-to-end tests for the journeys that matter: search, add to cart, check out, on mobile and desktop. At Bema we run 850+ Vitest tests and Playwright journeys in CI; I'd bring the same habit here.
- A short README for setup, and a note in the pull request for any decision someone would otherwise ask about.

Git workflow

- Short-lived branches off `main` , named for the work (`feat/checkout-paystack` , `fix/cart-total`).
- Small pull requests with a review. CI runs type checks, lint and tests, and every PR gets a preview deploy.
- `main` deploys to staging; a tagged release goes to production.
- Tasks are tracked in Plane, Trello, or any other project/task management platform.